In this contest, the following rules apply regarding the use of generative AI.

AtCoder Heuristic Contest Generative AI Usage Rules - Version 20250616 (https://info.atcoder.jp/entry/ahc-llm-rules-en)

For details, please see **this post (https://atcoder.jp/posts/1495)**.

Additionally, if you use generative AI in this contest, you must enter the instructions specified in the problem statement at the beginning of each chat, or configure them as custom instructions, project instructions, or in an instruction file that is automatically loaded by the generative AI tool.

A - Picnic

Time Limit: 2 sec / Memory Limit: 1024 MiB

Story

Nihonbashi Park is a park popular among visitors, boasting beautiful lawns and rich ponds. Efficiently arrange the crowds of visitors within the park to earn as much in usage fees as possible.

Problem Statement

There is a park represented by an $N \times N$ grid, where each cell is either lawn or pond. Let the top-left cell be $(0, 0)$, and let (x, y) denote the cell reached by moving x cells down and y cells right from there.

M groups who want to have a picnic come to the park. The groups are numbered $0, 1, \ldots, M-1$, and they arrive at the park in order of their numbers. Each group i has an arrival time S_i, a departure time T_i ($S_i < T_i$), a number of people P_i, and a base payment V_i.

You are the manager of this park, and for each group you decide whether to accept it and assign it a region, or to reject it. If you accept a group, you earn a usage fee from it. Your objective is to maximize your final amount of money.

Your money is initially 0. An accepted group occupies its assigned region until it departs. A group that is occupying a region is said to be "active".

Flow of Processing

Each time a group i arrives, the following steps are performed in order.

1. If $i > 0$, any group j that was accepted and satisfies $S_{i-1} < T_j < S_i$ has departed from the park at some point between the processing of group $i-1$ and now, and its region has been released and is now empty.
    - For each departed group, the usage fee is added to your money. The calculation of the usage fee is described later.
2. (Optional) You may move some of the active groups to other locations within the park by paying a cost. Your money decreases by the amount of the cost paid. Your money may become negative.
3. You decide whether to accept group i and assign it a region within the park, or to reject it. If you assign a region, you decide which region it occupies.
    - The assigned region is occupied by group i until time T_i. However, if group i is moved later, the region it occupies changes according to the move.

After performing the above steps for the last group $M-1$, all remaining active groups depart, and for each of them the usage fee is added to your money.

Conditions on the Assigned Region

The region assigned to group i must be a set of P_i lawn cells satisfying the following conditions.

- It is connected in the four directions (up, down, left, right).
    - That is, you can travel between any two cells of the region by repeatedly moving up, down, left, or right within the region.
- It does not contain any cell occupied by another group that is active at that time.

Moving Groups

Before deciding whether to accept or reject an arriving group, you may move active groups to other locations within the park. Each time you move a group j, your money decreases by $\max(\mathrm{round}(V_j \times R), 1)$.

When moving multiple groups together, the move is performed simultaneously in the following manner: **all the groups being moved first vacate their regions, and then each of them is placed at its specified position**. Because of this, moves such as swapping the positions of two groups are also possible. Each region after the move must also satisfy the "Conditions on the Assigned Region" (P_j lawn cells are connected, and do not overlap with any other active group in the state after the move).

Compactness and Usage Fee

The more compact the assigned region is, the more the group pays in usage fees. The compactness C is defined as follows.

- For a region of P cells, if the total length of its boundary with the outside of the region and with the outside of the park is L, the compactness C is defined as $C = \frac{4\sqrt{P}}{L}$.

We have $0 < C \leq 1$. The closer the region is to an axis-aligned square, the closer C is to 1; the more elongated the shape or the more holes it has, the smaller C becomes.

When an active group i departs, letting C_i be the compactness of the group, $\mathrm{round}(V_i \times C_i)$ is added to your money as a usage fee. You cannot earn a usage fee from a rejected group.

Here, C_i is the minimum compactness over all positions the group occupied during its stay (the position where it was initially placed, and, if it was moved, each position after a move).

For example, suppose a group with $P_i = 4$ is initially placed in a 2×2 square (boundary length $L = 8$, so $C = 1$), and is then moved to a 1×4 elongated shape ($L = 10$, so $C = 0.8$). In this case, C_i takes the minimum of the two, so $C_i = 0.8$.

Scoring

Let X be your amount of money at the point when the departure times of all groups have passed. Then $\max(X, 0)$ is the absolute score. The higher the absolute score, the better.

For each test case, we compute the **relative score** $\mathrm{round}(10^9 \times \left(\frac{\mathrm{YOUR}}{\mathrm{MAX}}\right))$, where YOUR is your absolute score and MAX is the maximum absolute score among all competitors obtained on that test case. The score of the submission is the sum of the relative scores.

The final ranking will be determined by the system test with more inputs which will be run after the contest is over. In both the provisional/system test, if your submission produces illegal output or exceeds the time limit for some test cases, only the score for those test cases will be zero, and your submission will be excluded from the MAX calculation for those test cases. The system test will be performed only for **the last submission which received a result other than** CE. Be careful not to make a mistake in the final submission.

Number of test cases

- Provisional test: 50
- System test: 2000. seeds.txt (https://img.atcoder.jp/ahc069/seeds.txt) (sha256=`f63752a346a388e810574d22c9ee271f4f77d0b5f9e353084f2c335417c96482`) will be published after the contest is over.

About the relative evaluation system

In both the provisional/system test, the standings will be calculated using only the last submission which received a result other than CE. Only the last submissions are used to calculate the MAX for each test case when calculating the relative scores.

The scores shown in the standings are relative, and whenever a new submission arrives, all relative scores are recalculated. On the other hand, the score for each submission shown on the submissions page is the sum of the absolute score for each test case, and the relative scores are not shown. In order to know the relative score of submission other than the latest one in the current standings, you need to resubmit it. If your submission produces illegal output or exceeds the time limit for some test cases, the score shown on the submissions page will be 0, but the standings show the sum of the relative scores for the test cases that were answered correctly.

About execution time

Execution time may vary slightly from run to run. In addition, since system tests simultaneously perform a large number of executions, it has been observed that execution time increases by several percent compared to provisional tests. For these reasons, submissions that are very close to the time limit may result in TLE in the system test. Please measure the execution time in your program to terminate the process, or have enough margin in the execution time.

Input and Output

This problem is an interactive task.

First, the parameters of the problem and the layout of the park are given from Standard Input.

$$
\begin{aligned}
&N\ M\ R\\
&\mathrm{row}_0\\
&\vdots\\
&\mathrm{row}_{N-1}
\end{aligned}
$$

- The first line: the size of the park N, the number of groups M, and the move cost coefficient R.
- The following N lines: the layout of the park. Each line is a string row_i of length N consisting of `.` (lawn) and `#` (pond). The j-th character of row_i indicates whether cell (i, j) is lawn or pond.

Then, for the M groups in order of arrival, the following "Input of Group Information", "Output of Moves", and "Output of Handling the Arrived Group" are repeated.

After each output, you must print a newline and then flush Standard Output. Otherwise, it may result in TLE.

Input of Group Information

The information of the arrived group is given in one line (for i in the order $0, 1, \ldots, M-1$).

$$
i\ S_i\ T_i\ P_i\ V_i
$$

Output of Moves

First, output the number of groups to move, A_i. The value of A_i is at least 0 and at most the number of currently active groups; output `0` if you do not perform any move.

$$
A_i
$$

Then, for each group to move, output the group number j and the coordinates of the P_j cells after the move, $(z_0, w_0), \ldots, (z_{P_j-1}, w_{P_j-1})$, in the following format. Repeat this A_i times.

$$
\begin{aligned}
&j\\
&z_0\ w_0\\
&\vdots\\
&z_{P_j-1}\ w_{P_j-1}
\end{aligned}
$$

You must not specify the same group more than once within a single move.

Output of Handling the Arrived Group

If you accept the arrived group, output `Yes` followed by the coordinates of the P_i cells to assign, $(x_0, y_0), \ldots, (x_{P_i-1}, y_{P_i-1})$, in the following format.

```
Yes
x_0 y_0
⋮
x_{P_i-1} y_{P_i-1}
```

If you reject it, output `No`.

```
No
```

View an example (https://img.atcoder.jp/ahc069/AdcJXWH4.html?lang=en&seed=0&output=sample)

Constraints

- $N = 50$
- $M = 1000$
- $0.001 \leq R \leq 0.1$
- $0 \leq S_i < T_i \leq 100000$
- $S_i < S_{i+1}\ (0 \leq i < M - 1)$
- The $2 \times M$ values $S_0, S_1, \ldots, S_{M-1}, T_0, T_1, \ldots, T_{M-1}$ are all distinct.
- $4 \leq P_i \leq 150$
- $0 < V_i \leq 10^8$
- S_i, T_i, P_i, V_i are integers.
- R is given with 3 decimal places.

Sample Program (Python)

▼ Details

Here is an example solution in Python. This program scans lawn cells that are not currently in use from the top row downward, and from the first such cell found, performs a breadth-first search; if it can visit P_i cells, it assigns them to the group. If a connected region of P_i cells cannot be found, it rejects the group.

```python
import math
import sys
from collections import deque

DIRS = [(-1, 0), (1, 0), (0, -1), (0, 1)]


class GroupInfo:
    def __init__(self, s, t, p, v):
        self.s = s        # arrival time
        self.t = t        # departure time
        self.p = p        # number of people
        self.v = v        # base payment
        self.c = None     # compactness (set after placement)
        self.pos = None   # coordinates of occupied cells [(row, col), ...] (set after placement)


def find_region(grid, owner, n, p):
    """
    Scan from the top row downward for the first empty lawn cell, then collect p cells from there using BFS.
    Return the list of coordinates if p cells can be collected, otherwise return None.
    """

    # Find the first empty lawn cell
    start = None
    for x in range(n):
        for y in range(n):
            if grid[x][y] == "." and owner[x][y] == -1:
                start = (x, y)
                break
        if start is not None:
            break
    if start is None:
        return None

    # BFS from start. Add each dequeued cell to the region, and stop when p cells are reached.
    visited = [[False] * n for _ in range(n)]
    visited[start[0]][start[1]] = True
    queue = deque([start])
    region = []
    while queue and len(region) < p:
        x, y = queue.popleft()
        region.append((x, y))
        for dx, dy in DIRS:
```

```
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and not visited[nx][ny] \
                    and grid[nx][ny] == "." and owner[nx][ny] == -1:
                visited[nx][ny] = True
                queue.append((nx, ny))

    return region if len(region) == p else None


def compactness(region, p):
    """Return the compactness."""
    cells = set(region)
    perimeter = 0
    for (x, y) in region:
        for dx, dy in DIRS:
            if (x + dx, y + dy) not in cells:
                perimeter += 1
    return 4 * math.sqrt(p) / perimeter


def main():
    n, m, r = input().split()
    n, m = int(n), int(m)
    grid = [input() for _ in range(n)]

    # owner[x][y]: the group number occupying that cell. -1 if empty.
    owner = [[-1] * n for _ in range(n)]
    # all groups that have arrived so far
    groups = []

    for i in range(m):
        _gi, s, t, p, v = map(int, input().split())
        group = GroupInfo(s, t, p, v)
        groups.append(group)

        # Release groups whose departure time is before the current time s
        for g in groups:
            if g.pos is not None and g.t < s:
                for (x, y) in g.pos:
                    owner[x][y] = -1
                g.pos = None

        # Do not perform any move
        print(0)

        region = find_region(grid, owner, n, p)
        if region is not None:
            # Assign: update owner, and record the compactness and occupied cells
```

```
            for (x, y) in region:
                owner[x][y] = i
            group.pos = region
            group.c = compactness(region, p)
            print("Yes")
            for (x, y) in region:
                print(x, y)
        else:
            # Reject
            print("No")
        sys.stdout.flush()


if __name__ == "__main__":
    main()
```

Input Generation

Let $\mathrm{rand}(x, y)$ be a function that returns a uniform random integer between x and y, inclusive, and let $\mathrm{randf}(x, y)$ be a function that returns a uniform random real number between x and y, inclusive. Also, let $\mathrm{gauss}(\mu, \sigma)$ be a function that returns a value from a Gaussian distribution with mean μ and standard deviation σ.

Generation of R

Let $R = \mathrm{rand}(1, 100) \times 0.001$.

Generation of the Park Shape

First, make all cells of the park lawn.

Let $\mathrm{num_cluster} = \mathrm{round}(2^{\mathrm{randf}(1.0, 8.0)})$, and choose $\mathrm{num_cluster}$ distinct cells uniformly at random and change them to pond.

Let $\mathrm{num_pond} = \mathrm{rand}(0, 900 - \mathrm{num_cluster})$, and repeat the following operation $\mathrm{num_pond}$ times.

- Among the lawn cells that are adjacent to a pond cell, choose one uniformly at random and change it to pond.

Note that in the generated park, not all lawn cells are necessarily connected in the four directions.

Generation of Group Information

Let $P_i = \mathrm{round}(\mathrm{randf}(2.0, \sqrt{150.0})^2)$.

S_i, T_i are determined as follows.

1. Let $\theta = \mathrm{rand}(2000, 8000)$.
2. For each i satisfying $0 \leq i < M$, in order, determine s_i, t_i as follows.
    1. Let l_i be the value generated from an exponential distribution with $\lambda = 1/\theta$, rounded to the nearest integer.
    2. If $l_i \geq 100000$, regenerate l_i and redo the subsequent steps for this i.
    3. Let $s_i = \mathrm{rand}(0, 100000 - 1 - l_i)$.
    4. Let $t_i = s_i + l_i + 1$.
    5. If s_i or t_i equals any already generated s_j or t_j $(0 \leq j < i)$, regenerate l_i and redo the subsequent steps for this i.
3. Let $(S_0, T_0), (S_1, T_1), \ldots, (S_{M-1}, T_{M-1})$ be (s_i, t_i) $(0 \leq i < M)$ sorted in ascending order of s_i.

V_i is determined as follows.

1. Let $V_i = \mathrm{round}(P_i \times (T_i - S_i)^{0.9} \times 2^{\mathrm{gauss}(0.0, 0.8)})$.
2. If V_i is 0, let $V_i = 1$.

3. If $V_i > 10^8$, let $V_i = 10^8$.

Tools (Input generator and visualizer)

- Web version (https://img.atcoder.jp/ahc069/AdcJXWH4.html?lang=en): This is more powerful than the local version, providing animations.
- Local version (https://img.atcoder.jp/ahc069/AdcJXWH4.zip): You need a compilation environment of Rust language (https://www.rust-lang.org/).
    - Pre-compiled binary for Windows (https://img.atcoder.jp/ahc069/AdcJXWH4_windows.zip): If you are not familiar with the Rust language environment, please use this instead.

Please be aware that sharing visualization results or discussing solutions/ideas during the contest is prohibited.

Use of Generative AI

Under the current AtCoder Heuristic Contest Generative AI Usage Rules — Version 20250616 (https://info.atcoder.jp/entry/ahc-llm-rules-en), it is prohibited to have an AI agent execute a solution program and automatically repeat improvements based on the execution results.

However, with the latest generative AI systems, the distinction between conversational services such as ChatGPT and AI agents such as Codex has become increasingly blurred. Even conversational services may, without an explicit instruction from the user, internally generate test cases, execute a solution program, and automatically repeat improvements based on the execution results.

Some participants already explicitly instruct generative AI systems not to engage in such behavior. To ensure fairness among participants and to enforce the intent of the current rules, anyone using generative AI in this contest must provide the instruction below either at the beginning of each chat or through a custom instruction, project instruction, or instruction file automatically loaded by the relevant generative AI tool, such as `AGENTS.md` or `CLAUDE.md`.

```
I am currently participating in an AtCoder Heuristic Contest, and I will use this generative AI to assist in developing my solution.

When using this generative AI, the "AtCoder Heuristic Contest Generative AI Usage Rules - Version 20250616" apply.

https://info.atcoder.jp/entry/ahc-llm-rules-en

Most importantly, after running the solution program, you must not modify or improve the solution, its approach, or its code based on the execution results unless the user gives a new explicit instruction to do so.

You may run the solution program and report its execution results, logs, scores, or other observations. After reporting them, you must stop and wait for a new instruction from the user before making any improvement based on those results.

Here, "solution program" refers to any program created or being created for the purpose of solving this contest problem, regardless of whether it was created by the user or by generative AI, and regardless of whether it is still in progress or already complete.
```